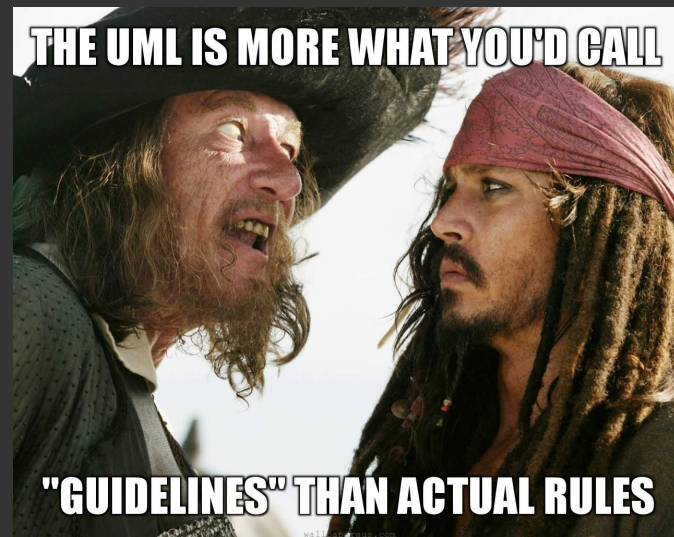


Structural UML Diagrams

And a bit of SOLID

Vladimir Soroka



Structural Diagrams

- Structural diagrams model the static aspects of a system

Diagram	Focus	Used For
Object	Object instances	Runtime examples, debugging
Package	Logical grouping	Large system organization
Component	Software modules	Architecture, interface definition
Deployment	Physical setup	System deployment & infrastructure

Object diagram

- **Definition:** Snapshot of instances and their relationships at a given time. Use case: Explains runtime behavior or debugging scenarios.
- **Notation:**
 - Underlined object names (e.g., :Order)
 - Attribute values shown
 - Links between objects (instances of associations)



Object Diagram Scenario

- A guest named **Alice** has made a booking at the Grand Hotel.
- Her booking includes two rooms: one single and one double.
- The booking is paid, and one of the rooms has a minibar with some items - soda for 4\$, chips for 3\$
- A receptionist named **Bob** handled the check-in.



Object diagram PlantUML

```
@startuml HotelBookingObjectDiagram
```

```
' Object Instances
```

```
object "Alice : Guest" as Guest1 {  
    name = "Alice"  
}
```

```
object "Booking1 : Booking" as Booking1 {  
    status = "Paid"  
    date = "2025-05-06"  
}
```

```
object "101 : Room" as Room1 {  
    type = "Single"  
}
```

```
object "102 : Room" as Room2 {  
    type = "Double"  
}
```

```
object "Bob : Receptionist" as Receptionist {  
    name = "Bob"  
}
```

```
object "Minibar102 : Minibar" as Minibar {  
    roomID = 102  
}
```

```
object "Soda : MinibarItem" as Item1 {  
    price = 4  
}
```

```
object "Chips : MinibarItem" as Item2 {  
    price = 3  
}
```

```
' Associations (Links)
```

```
Guest1 -- Booking1 : made
```

```
Booking1 -- Room1 : includes
```

```
Booking1 -- Room2 : includes
```

```
Receptionist -- Booking1 : checked in
```

```
Room2 -- Minibar : has
```

```
Minibar -- Item1 : contains
```

```
Minibar -- Item2 : contains
```

```
@enduml
```

Brought to you by

[PlantUML](https://plantuml.com/)

Package Diagram

- **Definition:** Groups classes into packages; shows dependencies between them.
- **Use case:** Organizing large systems, identifying layers/modules.
- **Notation:**
 - Rectangles with tabs = packages
 - Dashed arrows = dependencies

Package services be like: Your package will arrive within 0-12 hours, please stay home the entire day and answer the door within 30 seconds



Package Diagram Scenario

- Online Learning Platform
- The system allows students to register, enroll in courses, watch videos, and take quizzes. Instructors can create and manage courses. An admin can moderate content and manage users.

"Luke, you must learn the ways of the force"



"I'm ready, Obi Wan."



"Ooooookay. Let's see here. After you've logged in, you're gonna want to go to the student portal and click Jedi...."



Package diagram PlantUML

```
@startuml OnlineLearningPlatformPackages
```

```
package "auth" {  
    class LoginService  
    class RegistrationController  
}
```

```
package "user" {  
    class Student  
    class Instructor  
}
```

```
package "course" {  
    class CourseManager  
    class QuizEngine  
}
```

```
package "content" {  
    class VideoService  
}
```

```
package "admin" {  
    class AdminPanel  
}
```

```
package "utils" {  
    class EmailValidator  
    class Logger  
}
```

```
' Dependencies  
course --> auth : uses  
course --> user : manages users' courses  
course --> utils : logging, validation
```

```
content --> course : tied to course structure
```

```
admin --> user : manages users  
admin --> course : moderates content  
admin --> content : reviews videos
```

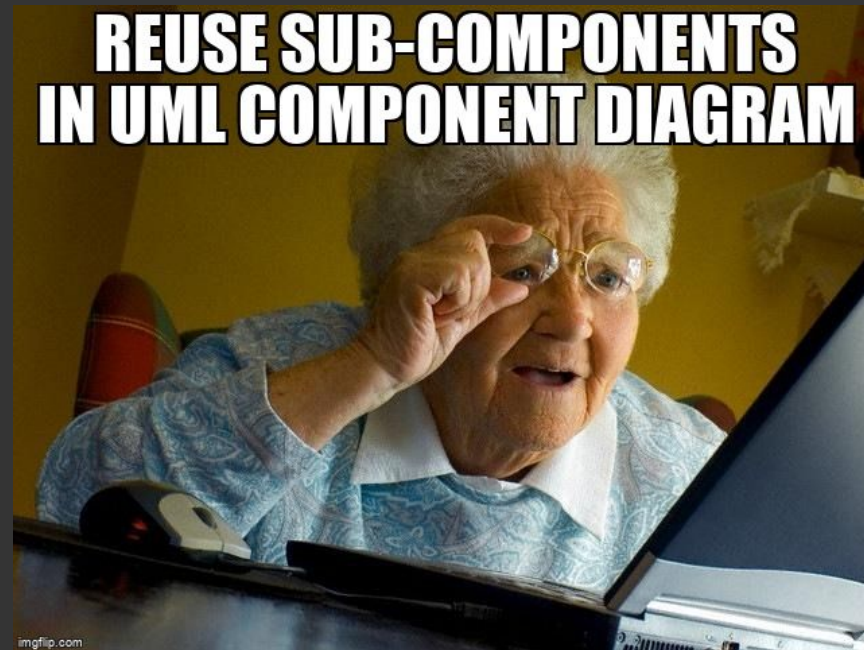
```
auth --> utils : validation, logging
```

```
@enduml
```

Brought to you by
[PlantUML](https://plantuml.com)

Component Diagram

- **Definition:** Models software components and their interfaces.
- **Use case:** Architecture design, interface contracts.
- **Notation:**
 - Rectangle with lollipop/socket for provided/required interfaces
 - Solid lines for connections

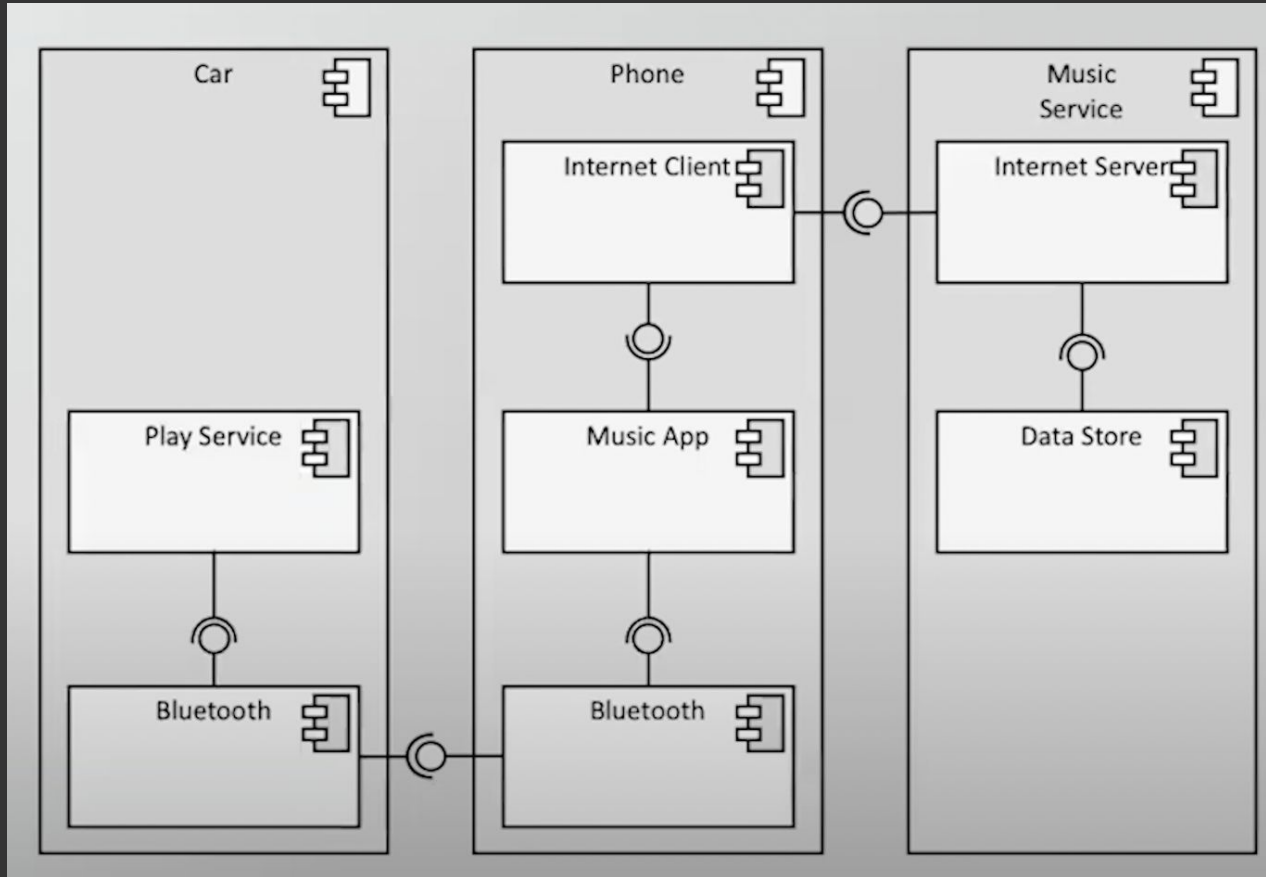


Component Diagram Scenario

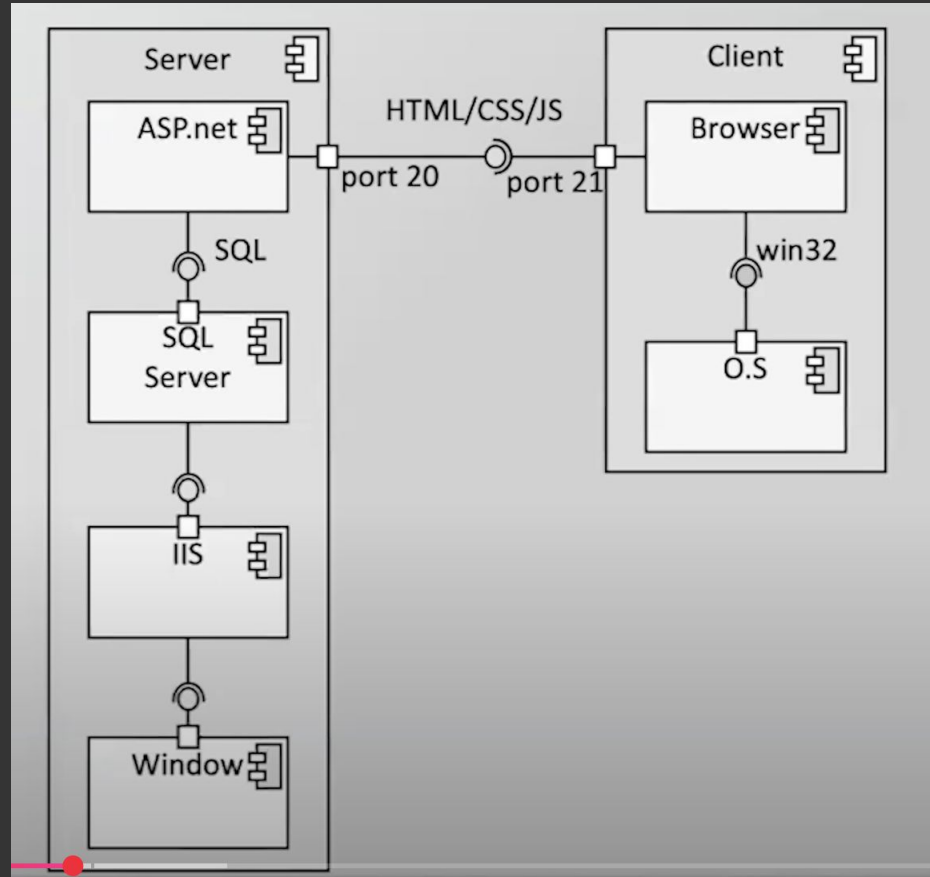
- Music Streaming in the Car
- A user streams music to her phone and then gets into her car and connects the audio to the car sound system



Component Diagram - Implementation



Component Diagram - Implementation



Deployment Diagram

- **Definition:** Shows physical deployment of software on hardware nodes. Use case: Infrastructure planning, DevOps, performance modeling.
-
- **Notation:**
 - 3D boxes for nodes
 - Artifacts shown inside nodes
 - Lines for communication paths



Deployment Diagram Scenario

- A system like Netflix consists of various software components deployed across different physical and virtual machines. The deployment diagram will show how components like web UI, media servers, databases, and content delivery are mapped to nodes.



Component Diagram PlantUML

```
@startuml EnhancedVideoStreamingDeployment
```

```
' Devices and main nodes
```

```
node "Client Device" <<mobile>> {  
}
```

```
node "User's Smart TV" <<tv>> {  
}
```

```
node "Web Server (EC2)" <<cloud>> {  
    node "Docked Container: Nginx" <<container>> {  
        artifact "WebUI.war"  
    }  
}
```

```
node "Docked Container: Backend JVM" <<container>> {  
    component "UserService"  
    component "AuthService"  
    artifact "UserService.jar"  
    artifact "AuthService.jar"  
}
```

```
node "Media Server (EC2)" <<cloud>> {  
    node "Docked Container: Media Engine"  
    <<container>> {  
        component "StreamingService"  
        component "Transcoder"  
    }  
}
```

```
node "CDN Node (Physical)" <<edge>> {  
    artifact "VideoCache"  
    component "CDNDistribution"  
}
```

```
node "Database Server (RDS)" <<database>> {  
    database "UserDB"  
    database "VideoMetadataDB"  
}
```

```
' Communication paths
```

```
"Client Device" --> "Web Server (EC2)" : HTTPS
```

```
"User's Smart TV" --> "Web Server (EC2)" : HTTPS
```

```
"Web Server (EC2)" --> "Media Server (EC2)" : gRPC
```

```
"Web Server (EC2)" --> "Database Server (RDS)" : JDBC
```

```
"Media Server (EC2)" --> "CDN Node (Physical)" : Internal  
Push
```

```
"Client Device" --> "CDN Node (Physical)" : HLS/HTTP
```

```
"User's Smart TV" --> "CDN Node (Physical)" : HLS/HTTP
```

```
@enduml
```

Brought to you by

PlantUML

Structural Diagrams - Summary

- **Object Diagrams** are like photographs of the system in action
- **Package Diagrams** help keep complexity organized
- **Component Diagrams** highlight contracts and boundaries
- **Deployment Diagrams** show how software lives in the real world

Diagram	Focus	Used For
Object	Object instances	Runtime examples, debugging
Package	Logical grouping	Large system organization
Component	Software modules	Architecture, interface definition
Deployment	Physical setup	System deployment & infrastructure

Introduction to SOLID



What is SOLID?

- SOLID is an acronym for five key principles of object-oriented design, introduced by Robert C. Martin ("Uncle Bob").

Letter	Principle	Focus
S	Single Responsibility Principle	Class should do one thing
O	Open/Closed Principle	Open for extension, closed for modification
L	Liskov Substitution Principle	Subtypes must be substitutable
I	Interface Segregation Principle	Many small interfaces > one big one
D	Dependency Inversion Principle	Depend on abstractions, not concrete classes

Single Responsibility Principle (SRP)

- A class should have only **one** reason to change

Bad:

```
class Report {  
    void generate();  
    void print(); // printing is a different  
responsibility  
}
```

Good:

- ReportGenerator
- ReportPrinter

Open/Closed Principle (OCP)

- Software entities should be open for extension, but closed for modification.

✗ Bad:

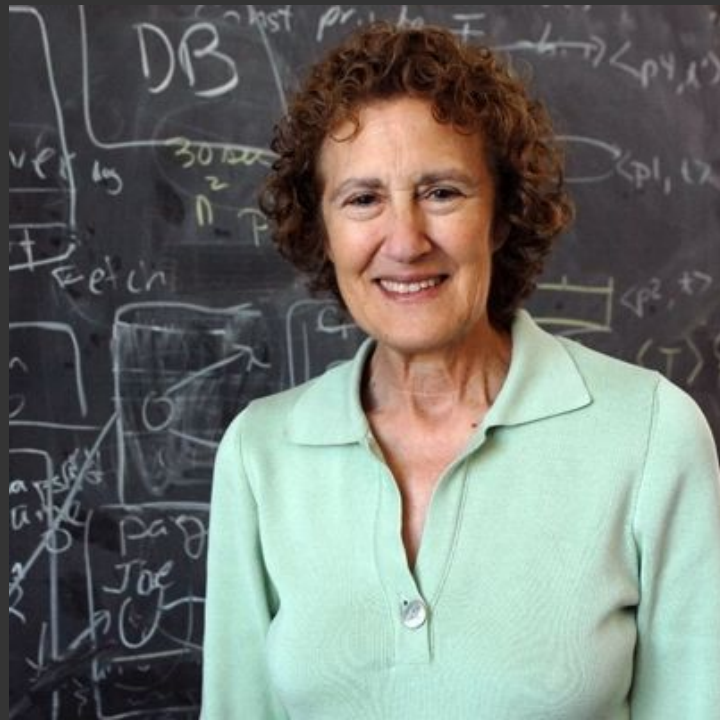
```
class DiscountService {  
    double apply(String type, double price) {  
        if (type.equals("CHRISTMAS")) return price * 0.9;  
        // more ifs  
    }  
}
```

✓ Good:

```
interface Discount { double apply(double price); }  
class ChristmasDiscount implements Discount { ... }  
class BlackFridayDiscount implements Discount { ... }
```

Liskov Substitution Principle (LSP)

- Barbara Liskov
- 🎓 Ph.D. in Computer Science from Stanford (1968) — the first woman in the U.S. to earn one!
- 🧠 Professor at MIT (Massachusetts Institute of Technology)
- 🛠️ Creator of CLU, one of the first programming languages to support abstract data types
- 🏆 Winner of the Turing Award (2008), often called the "Nobel Prize of computing"



Liskov Substitution Principle (LSP)

- Subtypes must be substitutable for their base types without altering correctness
 - Derived classes should not break the expectations of the base class.
 - Violations often occur when a subclass throws exceptions or changes behavior inappropriately.

✗ Violating LSP:

```
class Bird { void fly(); }  
class Ostrich extends Bird {  
    void fly() { throw new UnsupportedOperationException(); }  
}
```

✓ Fix:

```
interface Bird {}  
interface Flyable extends Bird { void fly(); }  
class Ostrich implements Bird {}  
class Sparrow implements Flyable { void fly(); }
```

Interface Segregation Principle (ISP)

- Clients should not be forced to depend on interfaces they do not use
- Break down large, general-purpose interfaces into smaller ones.

❌ Bad:

```
interface Worker {  
    void work();  
    void eat();  
}
```

Robots don't eat — this forces irrelevant implementation.

✅ Good:

```
interface Workable { void work(); }  
interface Feedable { void eat(); }  
class Human implements Workable, Feedable {}  
class Robot implements Workable {}
```

Benefit: Increases code clarity and adaptability.

Dependency Inversion Principle (DIP)

- Depend on abstractions, not on concretions
- High-level modules should not depend on low-level modules. Both should depend on abstractions.

❌ Bad:

```
class OrderService {  
    private PaypalService payment = new PaypalService();  
}
```

✅ Good:

```
interface PaymentService { void pay(); }  
class PaypalService implements PaymentService { ... }  
class OrderService {  
    private PaymentService payment;  
    OrderService(PaymentService p) { this.payment = p; }  
}
```

Benefit: Enables flexible substitution and unit testing.

